# تمرین سری چهارم

# برنامه نویسی پیشرفته

زمستان و بهار ۱۴۰۲-۱۴۰۳

دانشکدهٔ علوم ریاضی

## معرفی تمرین

هدف از این تمرین، تمیز کردن کد مطابق با اصول برنامه‌نویسی است. در این تمرین شما بایستی در کد بازی puzzle-8 که در اختیارتان قرار گرفته است تغییراتی ایجاد نمایید و قواعد اصلی تمیز زدن کد، خوانا بودن کد، توسعه‌پذیری کد، و همچنین اصول برنامه نویسی که در کلاس درس آموزش داده شده‌است را بر روی آن اعمال کنید.

## معرفی بازی

این بازی مشابه با بازی puzzle-15 طراحی شده که البته برای سادگی کار از جدولی با اندازهٔ ۳ در ۳ استفاده شده‌است. در صورتی که با این بازی آشنایی ندارید حتما با یکی از نسخه‌های آن چند بار بازی کنید.

- صفحهٔ ویکی‌پدیای بازی برای آشنایی بیشتر:

Puzzle-15 Wikipedia Page

- نسخه web بازی puzzle-15:

https://15puzzle.netlify.app

- نسخه Android بازی puzzle-15:

https://play.google.com/store/apps/details?id=com.jdapplications.puzzlegame&hl=en_US

- نسخه iOS بازی puzzle-15:

https://apps.apple.com/us/app/puzzle-15-modern/id1461109785

## توضیحات تمرین و کد نوشته شده

در این بازی، یک ترتیب اولیهٔ چینش برای اعداد در ابتدا توسط ما نظر گرفته می‌شود. از آنجایی که از هر وضعیت اولیه‌ای نمی‌توان به حالت برد رسید، یک تابع `solvable` در این کد پیاده‌سازی شده که با گرفتن وضعیت اولیه‌ی مهره‌ها، یک boolean بر می‌گرداند که تعیین می‌کند بازی قابل حل می‌باشد یا خیر.

اگر این boolean برابر با false باشد باید به کاربر یک خطا نشان دهیم.

دقت کنید که نیازی به فهمیدن نحوهٔ کار تابع `solvable` نیست و شما می‌توانید در کد خود از آن به شیوهٔ دلخواه استفاده کنید.

(در صورتی که علاقمند به فهمیدن منطق این تابع بودید اولین لینک داک که مربوط به Wikipedia می‌باشد، راجع به `solvable` بودن وضعیت‌ها توضیح داده است)

## کد منبع بازی

در کد منبع نوشته شده؛ تمام توابع بازی در یک فایل قرار دارد و یک بخش مهم از این تمرین، شکستن این فایل به کلاس‌های مورد نیاز و همچنین فایل‌های مختلف با وظایف مختلف است.

- کد منبع بازی را می‌توانید از آدرس زیر دریافت کنید:

*Sample Project - Puzzle 8*

## توضیح کلاس‌های موجود در کد منبع بازی

کلاس **Location**: وظیفهٔ این کلاس این است که به ازای هر مهرهٔ در بازی، موقعیت آن‌ها را ذخیره کند. طبیعی است که برای کشیدن تصویر این مهره‌ها در پنل، به همچین مختصاتی نیازمندیم.

کلاس **MyKeyListener**: این کلاس توابع KeyListener را implement می‌کند و به وسیلهٔ آن دکمه‌های مربوط به حرکت بازی را تنظیم می‌کنیم.

کلاس **PuzzlePiece**: این کلاس مشخصه‌های مهرهٔ بازی را نگه می‌دارد. تابع getClone برای این است که بتوانیم پس از فشردن دکمه، کپی‌ای از عکس‌ها در لحظه نگه داریم.

کلاس **MyPanel**: تابع paintComponent که بتوان به کمک آن صفحهٔ بازی را به صورت کلی کشید. تابع swapPieces نیز همانطور که گفته شد، عکس‌ها را جابه‌جا می‌کند که تابع paintComponent به درستی بکشد. این کلاس در واقع پنل اصلی بازی می‌باشد که یک آرایهٔ ۹ تایی از جنس `PuzzlePiece` به عنوان فیلد دارد.

## قابلیت‌های مورد نیاز بازی

اولین قابلیتی که باید اضافه کنید، یک فایل `config` است که باید در کنار سورس برنامهٔ شما قرار بگیرد و یک سری تنظیمات اولیه به جای hardcode شدن باید در آنجا قرار بگیرند.

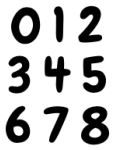
یک مثال از فایل `config` می‌تواند به این شکل باشد:

```
"initial-ordering": [0, 1, 2, 3 ,4 ,5 ,6 ,7, 8],
"images": ["0.png", "1.png", "2.png", "3.png", "4.png", "5.png", "6.png", "7.png", "8.png"]
```

- توجه کنید که فرمت فایل config می‌تواند به دلخواه شما تغییر کند و نیاز به استفاده از سینتکس یا حتی نام‌گذاری خاصی به ازای متغیر ها نیست ولی برای سادگی، از فرمت json استفاده شده است.

نقش config این است که برای هر کدام از خانه‌های این آرایهٔ ۹ تایی، تعیین کنیم که شمارهٔ عدد داخل این خانه چیست و همینطور باید آدرس این عکس‌ها را هم توسط config به برنامه بدهیم.

آرایهٔ initial-ordering ترتیب اولیهٔ این اعداد را نشان می‌دهد که در کد اولیه، این آرایه به صورت hardcode در آمده‌است.




- توجه: خانه‌های جدول، مطابق با شکل مقابل شماره‌گذاری شده‌اند و اعداد داخل initial-ordering با همین ترتیب روبرو درون جدول قرار خواهند گرفت.
- به عنوان مثال اگر عنصر هشتم initial-ordering برابر با عدد ۵ باشد، یعنی عدد دوم در سطر سوم جدول برابر با ۵ خواهد بود.

آرایهٔ images اسم عکس‌های استفاده شده را نشان می‌دهد. در واقع در این آرایه، به ترتیب اسم عکس‌های معادل با اعداد 0 تا 8 قرار خواهد گرفت. در کد اولیه نیز اسم عکس‌ها به صورت hardcode آمده‌است.

- در اینجا از ۹ عدد عکس استفاده شده‌است (در کد منبع، برای راحت تر شدن کار، خود مربع خالی هم عملا با عکس پیاده‌سازی شده و ساختار متفاوتی با بقیه‌ی خانه‌ها ندارد و همیشه در config عدد ۸ را متناظر با خانهٔ خالی می‌گیریم. به همین علت به جای ۸ عدد عکس، ۹ عدد عکس داریم!!)

- این صرفا یک مثال از config بود و شما می توانید با هر فرمتی که خواستید این کانفیگ را بنویسید.

دومین قابلیتی که از شما انتظار می‌رود در این کد پیاده کنید، این است که باید ابعاد بازی را در کد config ذخیره کنید که هرگاه نیاز به تغییر ابعاد داشتیم، به کمک فایل بتوان این امر را انجام داد. برای مثال:

```
"widthTiles": 3,
"heightTiles": 3,
```

نشان می‌دهد که بازی ما ۳ در ۳ است. باید با تغییر این فیلدها در config، بتوانیم اندازه بازی را گسترش دهیم.

برای گسترش بازی به ابعاد بالاتر، به تصاویری بیش از آنچه در فایل assets قرار داده‌شده نیاز پیدا خواهید کرد، که در این‌صورت پیشنهاد می‌شود که خودتان اعداد بعدی را (تا ۱۵ کافی‌است) دانلود کرده و در کنار فایل قرار دهید. بازی شما باید در تمام ابعاد (۴ در ۴ به پایین، اعم از حالات مستطیلی) به درستی کار کند و ممکن است نیاز به تغییراتی در منطق بازی داشته‌باشید.

سومین چیزی که از شما انتظار می‌رود در پروژه‌تان رعایت کرده باشید، جدا کردن تمامی اندازه‌ها و مقادیر به گونه‌ای که وابسته به مقدار داده شده نباشند. مثلا به جای اینکه در کل محیط پروژه، سایز پنل را با عدد وارد کنیم، با فیلد آن وارد می‌کنیم که هروقت خواستیم مقدار آن را تغییر بدیم، نیاز به تغییر در کل پروژه نباشد.

چهارمین موضوع، رندوم قرار دادن مکان عکس‌ها در جدول است. اینجا جایی است که تابع solvable به کمک شما می‌آید که تشخیص دهید یک بازی قابل حل است یا خیر.

آخرین چیزی که ملزم به پیاده‌سازی آن هستید، این است که در کد پروژه‌تان، بخشی تعبیه کنید که mode بازی را به دو قسمت تفکیک کند: ۱) حرکت قطری مجاز باشد. ۲) حرکت قطری مجاز نباشد.
دقت داریم که در صورت مجاز نبودن حرکت قطری، تابع solvable باید به درستی حل پذیر بودن بازی را (در حالت ۳*۳ و ۴*۴ کافی‌ست) بررسی کند.
حرکت قطری به حرکتی می‌گویند که علاوه بر ۴ جهت اصلی (بالا، پایین، چپ، راست) بتوانیم در ۴ جهت فرعی نیز (شمال شرقی، شمال غربی، جنوب شرقی، جنوب غربی) مهره‌ها را جابه‌جا کنیم.
نوع پیاده‌سازی این حرکت (اعم از اختصاص دکمه / موس) و دیگر مواردِ سلیقه‌ای بر عهده خودتان است.

## پایان بازی

- در نهایت، در حالتی که جدول ۳*۳ است بازی هنگامی تمام خواهد شد که اعداد 1 تا 8 به ترتیب صعودی در جدول قرار بگیرند.
  - در جدول با طول N و عرض M بازی زمانی پایان می‌یابد که که اعداد ۱ تا N*M-1 به ترتیب در جدول قرار گرفته باشند.
  - در کلاس `PuzzlePiece` یک متغیر داریم به اسم `pieceNumber` که نشان دهندهٔ عدد داخل عکس‌ها می‌باشد. در انتها برای اینکه ببینیم پازل حل شده یا نه، از همین متغیر استفاده می‌کنیم که ببینیم

آیا قطعات چیده شده، ترتیب درستی دارند یا خیر.

## امتیازی

در بخش امتیازی این تمرین، می‌خواهیم رابط کاربری‌مان در پروژه را تغییر دهیم.
از شما انتظار می‌رود که بخشی را طراحی کنید که با توجه به اصول **Clean Code** و بدون تغییر در منطق بازی، بتوانیم بازی را با گرافیک یا **CLI** انجام دهیم. کاربر باید در محیطی بتواند رابط کاربری‌ای را انتخاب و یا آن را تغییر دهد. منطق بازی در **CLI** باید به طور کاملاً صحیح پیاده، و اجرا شود.

## جمع‌بندی انتظارات

هدف از این تمرین بازنویسی (تمیز کردن) برنامهٔ نوشته شدهٔ فعلی است. در نسخه بازنویسی شدهٔ این تمرین، انتظار داریم که موارد زیر رعایت شده باشند:

- شکستن این فایل به کلاس‌های مورد نیاز و همچنین فایل‌ها و بسته‌های مختلف، با وظایف مختلف
- حداکثر سعی در رعایت اصول طراحی شی‌گرا و قواعد **S.O.L.I.D**
- وجود یک فایل `config` و استفاده از آن در کنار سورس کد برنامه، و قرار گرفتن یک سری تنظیمات اولیه در آن (به جای **hardcode** کردن)
- در کد منبع بازی، مفهومی به اسم جدول یا `Board` وجود ندارد! انتظار می‌رود که یک کلاس جدول طراحی کنید که با استفاده از اطلاعات این کلاس بتوانیم یک جدول را به طور کامل نمایش دهیم.
- همچنین انتظار می‌رود که یک کلاس طراحی کنید که بتواند با خواندن فایل `config`، مقادیر ثابت در بازی را نگه‌داری کند و در اختیار برنامه قرار دهد.
  - این کلاس باید با استفاده از دیزاین پترن `Singleton` طراحی شود.
  - رابط گرافیکی برنامه کاملا از منطق بازی جدا شده باشد به طوری که به راحتی بتوان:
    - بدون تغییر در منطق برنامه، رابط کاربری دیگری برای آن طراحی کرد.
    - رابط گرافیکی از جنس کنسول (ترمینال) برای برنامه طراحی کرد.

- پکیج‌بندی مناسب برنامه‌ها در پروژهٔ بازنویسی شده

برای یادآوری مباحث مربوط به کلین‌کد و دیزاین پترن، به این لینک مراجعه کنید.

**با آرزوی موفقیت**

**تیم درس برنامه‌نویسی پیشرفته**